



Tecnicatura Universitaria en Programación

Base de Datos II

Grupo 80

Trabajo Práctico Integrador

Sistema de Administración de Alquileres Temporarios

Profesores:

- *Simón, Angel*
- *Vélez, Laura*
- *Faure, Abel*
- *Lara Campos, Brian*

Alumno:

- Madero, Alejandro Martín (Legajo 30446)

Fecha de entrega límite: 17 de noviembre de 2025

Explicación del sistema

El sistema desarrollado tiene como objetivo administrar alquileres temporarios de propiedades entre anfitriones y huéspedes. A través de una aplicación web, los usuarios podrán publicar alojamientos, realizar reservas y gestionar pagos de manera sencilla y organizada.

Cada usuario puede tener diferentes roles dentro de la plataforma, pudiendo ser anfitrión, huésped o ambos. Los anfitriones publican sus propiedades indicando detalles como el título, la descripción, la capacidad, el precio por noche y la ubicación. Los huéspedes pueden buscar propiedades disponibles, realizar reservas y dejar una calificación una vez finalizada su estadía.

Las reservas vinculan a un huésped con una propiedad y registran las fechas de inicio y fin, el monto total y el estado de la operación. A su vez, los pagos se asocian a las reservas y registran el medio y la compañía utilizada. Finalmente, las calificaciones permiten dejar una puntuación y un comentario sobre cada alojamiento, ayudando a construir la reputación de los anfitriones y a mejorar la experiencia de los usuarios.

Funcionalidades principales

Los huéspedes podrán:

- Registrarse en el sistema y administrar su cuenta.
- Buscar propiedades por ciudad, fechas y cantidad de huéspedes.
- Realizar reservas y pagos en línea.
- Consultar su historial de estadías.
- Calificar las propiedades donde se alojaron y dejar comentarios.

Los anfitriones podrán:

- Registrar nuevas propiedades para ofrecer en alquiler.
- Editar o eliminar sus publicaciones.
- Ver las reservas recibidas y sus estados.
- Consultar estadísticas de ocupación y ganancias.
- Revisar las calificaciones y reseñas de sus huéspedes.

Los administradores del sistema podrán:

- Gestionar los usuarios, propiedades y reservas.

- Modificar los medios y compañías disponibles para realizar los pagos.
- Consultar estadísticas generales de uso y actividad.

Diagrama Entidad-Relación (DER):



Objetos de Base de Datos clave en el Sistema:

Vista 1 vw_reservasPorCiudad:

Esta vista muestra información estadística de las reservas agrupadas por ciudad, mostrando para cada destino el país correspondiente, los ingresos generados (excluyendo reservas canceladas), la cantidad de propiedades en el destino, el número de estadías finalizadas, confirmadas y canceladas, junto con el puntaje promedio de calificaciones y la duración promedio de las estadías en noches. Permite obtener una visión general del rendimiento turístico por ciudad dentro de la plataforma.

```
CREATE VIEW vw_reservasPorCiudad AS
SELECT
C.Nombre as Ciudad,
PA.Nombre as Pais,
SUM(CASE WHEN R.EstadoReserva NOT IN ('Cancelada') THEN R.Monto ELSE 0 END) AS IngresosGenerados,
count(r.idPropiedad) AS CantidadPropiedades,
SUM(CASE WHEN R.EstadoReserva = 'Finalizada' THEN 1 ELSE 0 END) as EstadiasFinalizadas,
SUM(CASE WHEN R.EstadoReserva = 'Cancelada' THEN 1 ELSE 0 END) AS ReservasCanceladas,
SUM(CASE WHEN R.EstadoReserva = 'Confirmada' THEN 1 ELSE 0 END) AS ReservasConfirmadas,
ROUND(AVG(CA.Puntuacion * 1.0), 2) AS CalificacionPromedio,
ROUND(AVG(DATEDIFF(DAY,R.FechaInicio, R.FechaFin) * 1.0), 2) PromedioNochesEstadia
FROM reservas R
INNER join Propiedades p on r.idPropiedad = P.idPropiedad
INNER JOIN ciudades C on p.idCiudad = C.idCiudad
INNER join paises PA on pa.idPais = C.idPais
LEFT JOIN Calificaciones CA on R.idReserva = ca.idReserva
GROUP BY C.Nombre, PA.Nombre
GO
```

Vista 2 vw_ReservasPendientesPorPropiedad:

Esta vista muestra, por cada propiedad, todas las reservas que aún están pendientes de pago, indicando el anfitrión responsable, su correo electrónico, la ciudad donde se encuentra el alojamiento, el monto total adeudado y la cantidad de reservas pendientes. Permite identificar rápidamente qué propiedades tienen pagos sin acreditar.

```
CREATE VIEW vw_ReservasPendientesPorPropiedad AS
SELECT
P.idPropiedad,
U.Nombre + ' ' + U.Apellido AS NombreAnfitrión,
u.Email AS EmailAnfitrión,
C.Nombre AS Ciudad,
SUM(R.Monto) AS MontoPendiente,
COUNT(R.idReserva) AS ReservasPendientesPago
FROM Reservas R
INNER JOIN Propiedades P ON R.idPropiedad = P.idPropiedad
INNER JOIN Ciudades C ON C.idCiudad = P.idCiudad
INNER JOIN Usuarios U ON U.idUsuario = P.idAnfitrión
where EstadoReserva = 'Pendiente'
GROUP BY P.idPropiedad,
U.Nombre + ' ' + U.Apellido,
U.Email,
C.Nombre
GO
```

Vista 3 vw_ingresosPorAnfitrión:

Esta vista resume el rendimiento de cada anfitrión mostrando su recaudación total, el ingreso promedio por estadía, la cantidad de propiedades que administra, el número total de reservas confirmadas y la duración promedio. Permite evaluar el desempeño económico de cada anfitrión dentro de la aplicación.

```
CREATE VIEW vw_ingresosPorAnfitrión AS
SELECT
u.idUsuario,
U.Nombre + ' ' + U.Apellido AS NombreApellido,
SUM(R.Monto) AS RecaudacionTotal,
SUM(r.Monto) / COUNT(r.idReserva) AS RecaudacionPromEstadia,
COUNT(DISTINCT P.idPropiedad) AS CantidadPropiedadesListadas,
COUNT(distinct r.idReserva) AS CantidadEstadias,
AVG(DATEDIFF(day, r.FechaInicio, r.FechaFin) * 1.0) as NochesPromEstadia
FROM Propiedades P
INNER JOIN Reservas R ON P.idPropiedad = R.idPropiedad AND R.ReservaConfirmada = 1
INNER JOIN USUARIOS U ON U.idUsuario = P.idAnfitrión
GROUP BY
U.idUsuario,
U.Nombre + ' ' + U.Apellido
GO
```

Stored Procedure 1 `sp_ListarPropiedadesDisponibles`:

Este procedimiento almacenado devuelve todas las propiedades activas disponibles para reservar en una ciudad determinada, con capacidad suficiente y sin reservas superpuestas dentro del rango de fechas solicitado.

```
CREATE PROCEDURE sp_ListarPropiedadesDisponibles(
    @Ciudad NVARCHAR(100),
    @Capacidad INT,
    @FechaInicio DATE,
    @FechaFin DATE
)
AS
BEGIN
    IF (@FechaFin <= @FechaInicio)
    BEGIN
        RAISERROR('La fecha de fin debe ser posterior a la fecha de inicio.', 16, 1);
        RETURN;
    END
    SELECT
        P.idPropiedad,
        U.idUsuario,
        U.Nombre,
        U.Apellido,
        P.Titulo,
        p.TipoPropiedad,
        P.PrecioNoche,
        DATEDIFF(DAY, @FechaInicio, @FechaFin) AS Noches,
        P.Capacidad,
        p.direccion,
        p.Descripcion,
        p.CalificacionPromedio,
        c.idCiudad AS idCiudad,
        C.Nombre AS Ciudad
    FROM Propiedades P
        INNER JOIN Ciudades C ON P.idCiudad = C.idCiudad
        INNER JOIN Usuarios U ON P.idAnfitrión = U.idUsuario
    WHERE C.Nombre LIKE @Ciudad
        AND P.Capacidad >= @Capacidad
        AND P.Activa = 1
        AND P.idPropiedad NOT IN (
            SELECT P.idPropiedad
            FROM Reservas R
            WHERE R.idPropiedad = P.idPropiedad
                AND (R.FechaInicio < @FechaFin)
                AND (R.FechaFin > @FechaInicio)
        )
END
```

Stored Procedure 2 `sp_ObtenerDetallePropiedad`:

Este procedimiento devuelve toda la información sobre una propiedad específica, incluyendo datos del anfitrión, ubicación, tipo de alojamiento, capacidad, precio por noche, calificación promedio y descripción. Se utiliza la información para ser mostrada a la hora de reservar.

```
CREATE PROCEDURE sp_ObtenerDetallePropiedad(  
    @idPropiedad INT  
)  
AS  
BEGIN  
    SELECT  
        P.idPropiedad,  
        U.idUsuario,  
        U.Nombre,  
        U.Apellido,  
        P.Titulo,  
        p.TipoPropiedad,  
        P.PrecioNoche,  
        P.Capacidad,  
        p.direccion,  
        p.Descripcion,  
        p.CalificacionPromedio,  
        C.idCiudad AS idCiudad,  
        C.Nombre AS Ciudad  
    FROM Propiedades P  
        INNER JOIN Ciudades C ON P.idCiudad = C.idCiudad  
        INNER JOIN Usuarios U ON P.idAnfitrión = U.idUsuario  
    WHERE P.idPropiedad = @idPropiedad  
END;
```

Stored Procedure 3 sp_realizarReserva:

Este procedimiento registra una nueva reserva calculando automáticamente el monto total según la cantidad de noches y el precio por noche de la propiedad. Inserta la reserva con estado “Pendiente” y marcada como no confirmada hasta la realización del pago.

```
CREATE PROCEDURE sp_realizarReserva (
    @idPropiedad INT,
    @FechaInicio DATETIME,
    @FechaFin DATETIME,
    @CantidadHuespedes INT,
    @idHuesped INT
)
AS
BEGIN
    BEGIN TRY
        DECLARE @CantidadNoches INT = DATEDIFF(DAY,@FechaInicio, @FechaFin)
        DECLARE @PrecioNoche DECIMAL(10,2)
        SELECT @PrecioNoche = PrecioNoche from Propiedades where idPropiedad = @idPropiedad;
        DECLARE @Monto DECIMAL(10,2) = @CantidadNoches * @PrecioNoche
        INSERT INTO Reservas(idPropiedad, idHuesped, FechaInicio, FechaFin, Monto, EstadoReserva, FechaCreacion, ReservaConfirmada, CantidadHuespedes)
        VALUES(@idPropiedad, @idHuesped, @FechaInicio, @FechaFin, @Monto, 'Pendiente', GETDATE(), 0, @CantidadHuespedes)

    END TRY
    BEGIN CATCH
        RAISERROR('No se pudo reservar la propiedad seleccionada', 16, 1);
    END CATCH
END
GO
```


Stored Procedure 4 sp_registrarPago:

Este procedimiento registra el pago de una reserva validando previamente que exista, que no haya sido abonada antes y que el monto ingresado coincida con el valor total. Si todo es correcto, genera un número de factura, inserta el pago como aprobado y actualiza la reserva marcándola como confirmada.

```
create PROCEDURE sp_registrarPago(
    @idReserva INT,
    @Monto DECIMAL(10,2),
    @idMedioPago INT,
    @idCompañia INT
)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION
        DECLARE @MontoReserva DECIMAL(10,2)
        DECLARE @EstadoReserva NVARCHAR(20)
        DECLARE @ReservaConfirmada BIT
        DECLARE @Factura NVARCHAR(50)

        SELECT
            @EstadoReserva = R.EstadoReserva,
            @ReservaConfirmada = R.ReservaConfirmada,
            @MontoReserva = R.Monto
        FROM Reservas R
        WHERE R.idReserva = @idReserva

        IF @MontoReserva IS NULL
        BEGIN
            RAISERROR('La reserva no existe.', 16, 1);
            RETURN
        END

        IF @ReservaConfirmada = 1
        AND @EstadoReserva = 'Confirmada'
        BEGIN
            RAISERROR('La reserva ya fue abonada', 16, 1)
            RETURN
        END

        IF @Monto <> @MontoReserva
        BEGIN
            RAISERROR('El monto no coincide con el valor de la reserva', 16, 1)
            RETURN
        END

        SET @Factura = 'FAC-' + FORMAT(GETDATE(), 'yyyyMMddHHmmssfff');

        INSERT INTO Pagos(idReserva, idMedioPago, idCompañiaPago, Monto, FechaPago, EstadoPago, NFactura)
        VALUES(@idReserva, @idMedioPago, @idCompañia, @Monto, GETDATE(), 'Aprobado', @Factura)

        UPDATE Reservas
        SET EstadoReserva = 'Confirmada',
            ReservaConfirmada = 1,
            FechaModificacion = GETDATE()
        WHERE idReserva = @idReserva;

        COMMIT TRANSACTION
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION
        DECLARE @ERROR NVARCHAR(1000) = ERROR_MESSAGE();
        RAISERROR(@ERROR, 16, 1);
    END CATCH
END
```

Stored Procedure 5 sp_calificarEstadia:

Este procedimiento permite calificar una estadia una vez que la reserva ha finalizado. Verifica que la fecha de salida ya haya pasado y que la reserva no haya sido calificada previamente. Si las condiciones se cumplen, registra la puntuación y el comentario, y actualiza el estado de la reserva a “Finalizada”.

```
CREATE PROCEDURE sp_calificarEstadia(  
    @idReserva INT,  
    @Comentario NVARCHAR(500),  
    @Puntaje INT  
)  
AS  
BEGIN  
    BEGIN TRY  
        BEGIN TRANSACTION  
        DECLARE @FechaActual DATETIME = GETDATE()  
        DECLARE @FechaFinalizacionEstadia DATETIME  
  
        SELECT @FechaFinalizacionEstadia = R.FechaFin  
        FROM Reservas R  
        WHERE R.idReserva = @idReserva  
  
        IF @FechaFinalizacionEstadia > @FechaActual  
        BEGIN  
            RAISERROR('La estadia no finalizo todavia, no es posible calificarla', 16, 1)  
            RETURN  
        END  
  
        IF EXISTS(SELECT 1 FROM Calificaciones WHERE idReserva=@idReserva)  
            RAISERROR('La reserva ya fue calificada.',16,1);  
  
        INSERT INTO Calificaciones(idReserva, Puntuacion, Comentario, FechaCalificacion)  
        VALUES(@idReserva, @Puntaje, @Comentario, GETDATE())  
  
        UPDATE Reservas  
        SET EstadoReserva = 'Finalizada',  
            FechaModificacion = GETDATE()  
        WHERE idReserva = @idReserva  
  
        COMMIT TRANSACTION  
    END TRY  
    BEGIN CATCH  
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION  
        DECLARE @ERROR NVARCHAR(1000) = ERROR_MESSAGE();  
  
        RAISERROR(@ERROR, 16, 1)  
    END CATCH  
END
```

Trigger 1 tr_desactivarPropiedades:

Este trigger se ejecuta automáticamente en la entidad Usuarios cuando se modifica el estado *Activo* de un usuario. Si el usuario es desactivado, todas sus propiedades se despublican y se registra la fecha de baja. Si vuelve a activarse, las propiedades se reactivan, poniéndolas nuevamente disponibles en la aplicación y registrando la fecha de alta.

```
CREATE TRIGGER tr_desactivarPropiedades on Usuarios
AFTER UPDATE
AS
BEGIN
    IF UPDATE(Activo)
    BEGIN
        DECLARE @idUsuario INT;
        Declare @Activo INT;
        SELECT @idUsuario = idUsuario, @Activo = Activo
        FROM inserted;

        IF(@Activo = 0)
        BEGIN
            UPDATE Propiedades
            SET Activa = 0,
            FechaBajaPublicacion = GETDATE()
            WHERE idAnfitrion = @idUsuario;
        END;

        IF(@Activo = 1)
        BEGIN
            UPDATE Propiedades
            SET Activa = 1,
            FechaAltaPublicacion = GETDATE(),
            FechaBajaPublicacion = NULL
            where idAnfitrion = @idUsuario
        END;
    END
END
```

Trigger 2 tr_PrevenirReservasSuperpuestas:

Este trigger se ejecuta en la entidad Reservas después de una inserción. Evita que se inserten reservas con fechas superpuestas para la misma propiedad. Si bien previamente el **sp_ListarPropiedadesDisponibles** mostró solamente las propiedades disponibles en las fechas solicitadas, en el transcurso de tiempo que el usuario haya tardado hasta decidir reservar, otra persona podría haberlo hecho para el mismo rango de fechas, con lo cual es necesario volver a verificar si las fechas siguen disponibles. Si detecta solapamiento, genera un error y revierte la transacción, impidiendo que la reserva inválida quede registrada.

```
CREATE TRIGGER tr_PrevenirReservasSuperpuestas ON Reservas
AFTER INSERT
AS
BEGIN
    DECLARE @FechaInicio DATETIME
    DECLARE @FechaFin DATETIME
    DECLARE @idPropiedad INT
    DECLARE @idReserva INT

    SELECT @FechaInicio = FechaInicio, @FechaFin = FechaFin, @idPropiedad = idPropiedad, @idReserva = idReserva FROM inserted

    IF EXISTS (
        SELECT TOP 1 R.idReserva FROM Reservas R
        WHERE R.idPropiedad = @idPropiedad
        AND R.idReserva <> @idReserva
        AND (R.FechaInicio < @FechaFin)
        AND (R.FechaFin > @FechaInicio)
        AND R.EstadoReserva NOT IN ('Cancelada')
    )
    BEGIN
        RAISERROR('Las fechas elegidas ya no estan disponibles', 16, 1)
        ROLLBACK TRANSACTION
    END
END
```

Trigger 3 tr_actualizarPuntajePromedio:

Este trigger recalcula automáticamente el puntaje promedio de una propiedad cada vez que se inserta una nueva calificación en la entidad Calificaciones. Obtiene la propiedad asociada a la reserva calificada, calcula el promedio de todas sus puntuaciones y actualiza el valor en la tabla Propiedades

```
CREATE TRIGGER tr_actualizarPuntajePromedio ON Calificaciones
AFTER INSERT
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION

        DECLARE @idReserva INT
        DECLARE @idPropiedad INT
        DECLARE @NuevoPuntaje DECIMAL(3,2)

        SELECT @idReserva = I.idReserva, @idPropiedad = r.idPropiedad from inserted I
        INNER JOIN Reservas R on i.idReserva = R.idReserva

        select @NuevoPuntaje = AVG(C.Puntuacion * 1.0) from Calificaciones C
        INNER JOIN Reservas R ON R.idReserva = C.idReserva
        WHERE R.idPropiedad = @idPropiedad

        UPDATE Propiedades
        SET CalificacionPromedio = @NuevoPuntaje
        WHERE idPropiedad = @idPropiedad

        COMMIT TRANSACTION
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        DECLARE @ERROR NVARCHAR(1000) = ERROR_MESSAGE();
        RAISERROR(@ERROR, 16, 1)
    END CATCH
END
```

Link al repositorio GitHub: <https://github.com/Alejandro-Madero/Sistema-Administraci-n-Alquileres-Temporarios>

Link al DER: <https://www.drawdb.app/editor?shareId=85097414e7bf75e60ed3e18b349b394a>

Link al video: <https://youtube.com/watch?v=LPITt4CoRZY>